Real-Time Sliding Window Analytics Engine: A Deque-Based Amortized O(1) Streaming Framework

Aaditya Diwan
25206991
Contribution: 50%
Peng Wu

25203407
Contribution: 50%
Advanced Data Structures

Spring 2025

February 26, 2026

1 Problem Domain Description

Modern computing systems increasingly operate in streaming environments where data arrives continuously and must be processed in real time. Applications such as financial tick processing, Internet of Things (IoT) sensor monitoring, intrusion detection systems, log analytics, and performance monitoring require efficient mechanisms to compute statistics over recent observations [1]. These statistics are typically evaluated over a fixed-size sliding window that represents the most recent W elements of a potentially unbounded data stream.

A fundamental challenge in streaming analytics is maintaining aggregate statistics efficiently without recomputing them from scratch after each new input. Naïve implementations incur linear time per update, which becomes computationally infeasible at scale. For instance, a system processing 1 million events per second with a window size of 1000 would require 10^9 operations per second using a straightforward approach—an unacceptable computational burden for latency-sensitive applications.

This project presents a **space-optimized real-time sliding window analytics engine** built using advanced Deque Abstract Data Type (ADT) principles. The system maintains sliding maximum and minimum statistics in amortized constant time while supporting dynamic window resizing. The design demonstrates how appropriate data structure selection fundamentally transforms algorithmic performance, achieving a theoretical improvement from $O(nW)$ to $O(n)$ total complexity for processing n elements.

1.1 Problem Statement

Given a stream of numeric inputs $S = \{x_1, x_2, \ldots, x_n\}$ and a window size W, at each time step $i \geq W$, we must compute:

$$\text{Max}(i) = \max\{x_{i-W+1}, \ldots, x_i\} \tag{1}$$

$$\text{Min}(i) = \min\{x_{i-W+1}, \ldots, x_i\} \tag{2}$$

The problem is subject to the following constraints:

(1) **Unbounded stream length**: The stream may be arbitrarily large, potentially exceeding available memory

(2) **Space efficiency**: Memory usage must remain bounded by $O(W)$, independent of stream length

(3) **Time efficiency**: Per-update time complexity should be constant on average

(4) **Dynamic adaptation**: The window size W may change dynamically at runtime without requiring complete reconstruction

These constraints reflect real-world requirements where streaming systems must operate continuously with bounded resources while maintaining responsiveness to configuration changes.

1.2 Motivation and Real-World Applications

Sliding window analytics underpins numerous mission-critical systems across diverse domains:

Financial Systems: High-frequency trading platforms compute rolling price maxima and minima to detect volatility spikes, identify arbitrage opportunities, and trigger stop-loss orders. A delay of even milliseconds can result in significant financial losses [5].

Sensor Networks: Industrial IoT deployments monitor thousands of sensors for threshold breaches. Efficient sliding window computation enables real-time anomaly detection, preventing equipment failures and safety incidents.

Cloud Infrastructure: Platform monitoring systems track CPU utilization, memory pressure, and network throughput over recent intervals. These metrics drive autoscaling decisions and capacity planning [6].

Cybersecurity: Intrusion detection systems analyze network traffic patterns over sliding windows to identify distributed denial-of-service attacks, port scanning, and other malicious activities.

Real-time AI Pipelines: Online machine learning systems extract features from streaming data for model inference and continuous learning. Efficient feature computation directly impacts system throughput and latency.

In large-scale systems where n may exceed 10^7 or higher, an inefficient implementation directly impacts latency, throughput, and system scalability. The difference between $O(W)$ and $O(1)$ amortized per-update complexity can mean the difference between a viable production system and one that cannot meet service-level objectives.

2 Theoretical Foundations of the Data Structure(s) Utilised

2.1 Why Stack/Queue/Deque ADTs Are Appropriate

The sliding window model exhibits two essential properties that guide data structure selection:

1. **FIFO expiration**: Elements expire in the same order they arrive—the oldest element leaves first
2. **Monotonic ordering requirement**: We need fast access to extrema while maintaining temporal ordering

An analysis of candidate Abstract Data Types reveals:

Table 1: ADT Suitability Analysis for Sliding Window Problem

ADT	Core Operations	Suitability
Stack	$O(1)$ push/pop at one end	LIFO behavior conflicts with FIFO window expiration requirement
Queue	$O(1)$ enqueue/dequeue	Supports FIFO but no mechanism for monotonic ordering
Deque	$O(1)$ **insert/delete both ends**	**Enables monotonic ordering while supporting expiration**

2.2 Why Deque Is Optimal

A Deque (Double-Ended Queue) is particularly well-suited for this problem because:

1. **Dual-ended access**: $O(1)$ insertion and deletion at both front and back enables both monotonic maintenance (back operations) and expiration handling (front operations)
2. **Monotonic ordering**: By removing elements from the back during insertion, we maintain a monotonically decreasing (for maximum) or increasing (for minimum) sequence of candidate extrema
3. **Efficient expiration**: Front operations allow removal of expired elements without affecting the monotonic property
4. **Memory efficiency**: Only stores indices of potential extrema, not all W elements

The key insight is that not all elements in the window are candidates for the current or future maximum/minimum. By maintaining only the "useful" elements in monotonic order, we achieve both space efficiency and constant-time query operations.

2.3 Algorithm Design

2.3.1 Naïve Baseline Approach

A direct approach maintains a list of the last W elements. For each new element:

Algorithm 1 Naïve Sliding Window Maximum

```
for each element x_i in stream do
    Append x_i to window
    if window size > W then
        Remove oldest element
    end if
    Scan entire window to find maximum
end for
```

Time complexity per update: $O(W)$

Total complexity for n elements: $O(nW)$

For $W = 1000$ and $n = 10^6$, this requires approximately 10^9 operations—clearly infeasible for real-time applications.

2.3.2 Monotonic Deque Algorithm

The core insight is to maintain a deque of indices such that the corresponding values form a monotonic sequence.

For maximum tracking:

- Values at stored indices are maintained in strictly decreasing order
- The front element always represents the current maximum

For minimum tracking:

- Values at stored indices are maintained in strictly increasing order
- The front element always represents the current minimum

Key observation: If a new element x_i is greater than x_j where $j < i$, then x_j can never be the maximum for any future window that includes x_i. This allows us to safely discard x_j.

Algorithm 2 Monotonic Deque Sliding Maximum

```
1: for each element at index i with value A[i] do
2:     // Remove expired elements
3:     while deque.front() ≤ i − W do
4:         deque.pop_front()
5:     end while
6:
7:     // Maintain monotonic property
8:     while deque is not empty and A[deque.back()] ≤ A[i] do
9:         deque.pop_back()
10:    end while
11:
12:    // Add current element
13:    deque.push_back(i)
14:
15:    // Query maximum
16:    return A[deque.front()]
17: end for
```

The minimum algorithm is symmetric, using $\geq$ instead of $\leq$ in line 7.

2.4 Complexity Analysis

2.4.1 Amortized Time Complexity

Key observation: Each element is inserted into the deque exactly once and removed at most once.

- Total insertions across n elements: n
- Total removals (expiration + monotonic maintenance): $\leq n$
- Total deque operations: $\leq 2n$

Total time complexity: $T(n) = O(n)$
Amortized per-update complexity: $O(n)/n = O(1)$

2.4.2 Worst-Case Analysis

Consider a strictly increasing input sequence: $1, 2, 3, \ldots, n$

When element i arrives, all previous elements ($i - 1$ elements) are removed from the deque back.

Worst-case single update: $O(W)$

However, each element is removed at most once across all updates, so: **Total across all updates**: $O(n)$

This demonstrates the distinction between worst-case single-operation complexity and amortized complexity. While individual operations may be expensive, the total cost amortized over all operations remains linear.

2.4.3 Space Complexity

- **Circular buffer**: $O(W)$ — stores exactly W values
- **Maximum deque**: $O(W)$ — stores at most W indices
- **Minimum deque**: $O(W)$ — stores at most W indices

Total memory: $O(W)$

Crucially, space complexity is independent of stream length n, enabling processing of arbitrarily large streams with bounded memory.

2.5 Comparison to Alternative Approaches

Table 2: Comparison of Sliding Window Algorithms

Approach	**Time**	**Space**	**Complexity**	**Notes**
Naïve scan	$O(W)$	$O(W)$	Low	Simple but impractical
Binary heap	$O(\log W)$	$O(W)$	Medium	Requires lazy deletion
Segment tree	$O(\log W)$	$O(W)$	High	Range query overhead
Self-balancing BST	$O(\log W)$	$O(W)$	High	Complex implementation
Monotonic deque	$O(1)$ **amort.**	$O(W)$	**Medium**	**Asymptotically optimal**

The monotonic deque approach achieves the best possible asymptotic complexity for this problem while maintaining reasonable implementation complexity and excellent practical performance through cache-friendly data structures.

3 Analysis/Design (UML Diagram(s))

3.1 System Architecture Overview

The implementation follows a modular architecture with clear separation of concerns. The system comprises four main components:

1. **SlidingWindowEngine**: Core streaming engine orchestrating window management
2. **MonotonicDeque**: Specialized deque maintaining monotonic ordering

3. **NaiveWindow**: Baseline implementation for performance comparison
4. **StreamGenerator**: Synthetic data generation for experimental validation

3.2 Class Diagram

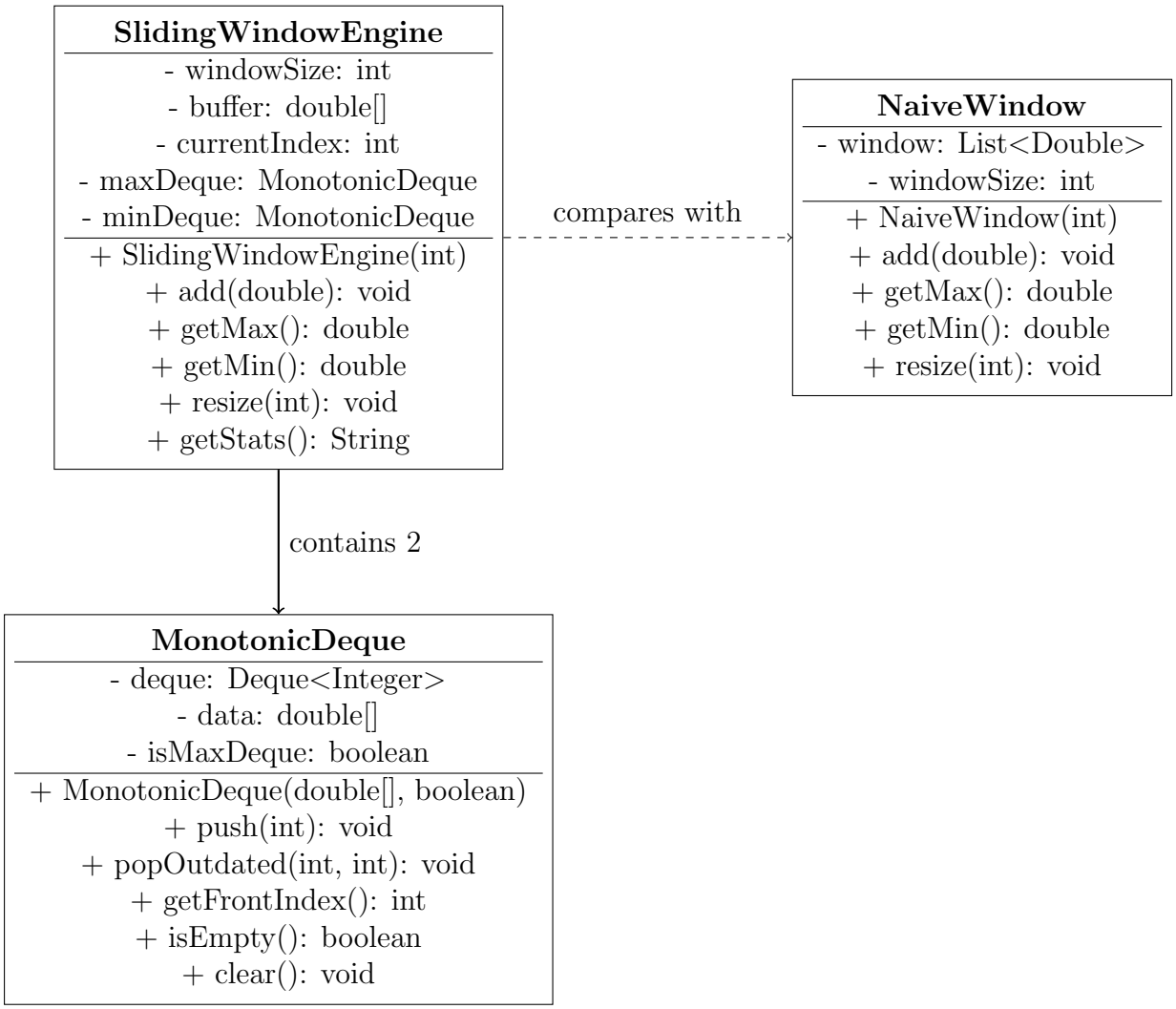

Figure 1: Class Diagram of Sliding Window Analytics Engine

3.3 Design Decisions and Rationale

3.3.1 Separation of Concerns

Decision: Create separate `MonotonicDeque` class rather than embedding logic in `SlidingWindowEngine`

Rationale:

- Promotes single responsibility principle
- Enables reuse for other extrema-tracking problems

- Simplifies testing of monotonic invariants
- Improves code maintainability and readability

3.3.2 Index-Based Storage in Deque

Decision: Store indices in deque rather than values directly.

Rationale:

- Enables efficient expiration checking (compare index with window boundary)
- Allows value lookup in $O(1)$ via circular buffer
- Reduces memory overhead (int vs double)
- Provides temporal information for debugging and validation

3.3.3 Circular Buffer for Data Storage

Decision: Use modulo arithmetic (`index % windowSize`) for buffer access.

Rationale:

- Eliminates costly array shifting operations
- Maintains $O(1)$ insertion regardless of stream length
- Optimal cache locality through contiguous memory
- Space-efficient: stores exactly W elements

3.3.4 Dual Deque Architecture

Decision: Maintain separate deques for maximum and minimum.

Rationale:

- Monotonic properties differ (decreasing vs increasing)
- Independent query operations without interference
- Cleaner API: `getMax()` and `getMin()` without parameters
- Parallelization potential for concurrent queries

3.3.5 ArrayDeque Implementation Choice

Decision: Use Java's `ArrayDeque` rather than `LinkedList`.

Rationale:

- Superior cache locality (contiguous array vs scattered nodes)
- Lower memory overhead (no node pointers)
- Faster iteration and access patterns
- Better constant factors despite identical asymptotic complexity

3.4 Sequence Diagram: Adding Elements

The following sequence diagram illustrates the interaction between components when adding a new element to the sliding window:

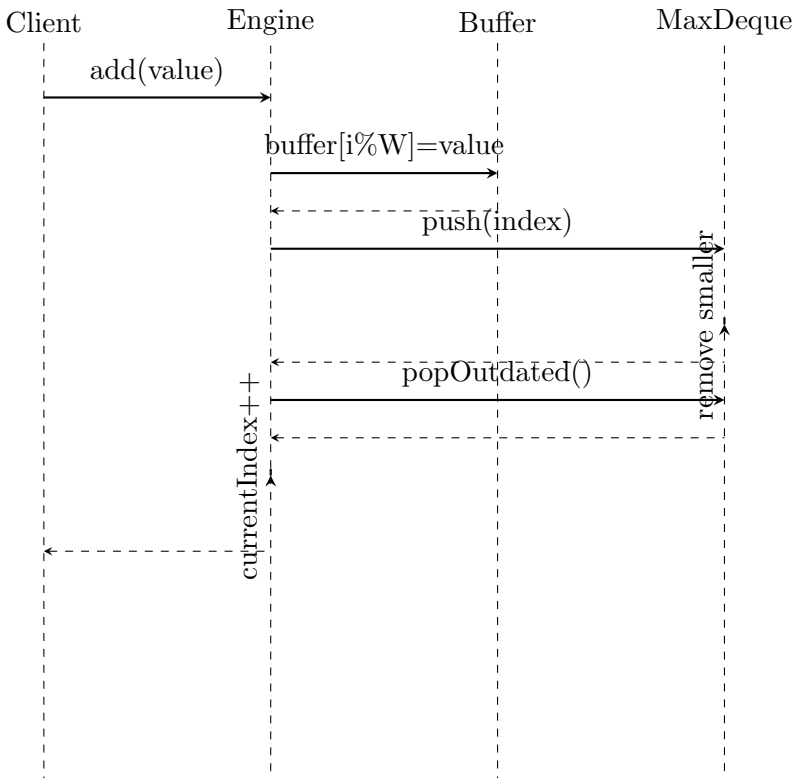

Figure 2: Sequence Diagram for add() Operation

4 Code Implementation

4.1 GitHub Repository

GitHub: `https://github.com/aaditya-diwan/sliding-window-analytics`

4.2 Implementation Overview

4.3 Core Data Structures

4.3.1 Circular Buffer Implementation

The circular buffer is implemented as a fixed-size array with modulo arithmetic for index calculation. This design eliminates the need for expensive element shifting operations.

```java
public class SlidingWindowEngine {
    private int windowSize;
    private double[] buffer;
    private int currentIndex;

    public SlidingWindowEngine(int windowSize) {
        this.windowSize = windowSize;
        this.buffer = new double[windowSize];
        this.currentIndex = 0;
    }

    public void add(double value) {
        // Compute circular buffer index to avoid array shifting
        int bufferIndex = currentIndex % windowSize;

        buffer[bufferIndex] = value;

        maxDeque.push(bufferIndex);
        minDeque.push(bufferIndex);

        maxDeque.popOutdated(currentIndex, windowSize);
        minDeque.popOutdated(currentIndex, windowSize);

        currentIndex++;
    }
}
```

Listing 1: Circular Buffer Implementation in SlidingWindowEngine.java

Key Implementation Details:

- **Modulo arithmetic**: `bufferIndex = currentIndex % windowSize` ensures proper wrapping
- **No shifting**: Overwrites oldest element directly, avoiding $O(W)$ shift cost
- **Cache-friendly**: Contiguous memory layout improves CPU cache utilization

4.3.2 Monotonic Deque Implementation

The `MonotonicDeque` class maintains indices in strictly monotonic order to enable $O(1)$ extrema queries.

```java
public class MonotonicDeque {
    private Deque<Integer> deque = new ArrayDeque<>();
    private double[] data;
    private boolean isMaxDeque;

    public MonotonicDeque(double[] data, boolean isMaxDeque) {
        this.data = data;
        this.isMaxDeque = isMaxDeque;
    }

    public void push(int index) {
        if (isMaxDeque) {
            while (!deque.isEmpty() &&
                   data[deque.peekLast()] <= data[index]) {
                deque.pollLast();
            }
        } else {
            while (!deque.isEmpty() &&
                   data[deque.peekLast()] >= data[index]) {
                deque.pollLast();
            }
        }
        deque.offerLast(index);
    }

    public void popOutdated(int currentIndex, int windowSize) {
        if (!deque.isEmpty() &&
            deque.peekFirst() <= currentIndex - windowSize) {
            deque.pollFirst();
        }
    }

    public int getFrontIndex() {
        return deque.peekFirst();
    }

    public boolean isEmpty() {
        return deque.isEmpty();
    }
}
```

Listing 2: MonotonicDeque Core Implementation in MonotonicDeque.java

Invariants Maintained:

1. **Temporal ordering**: Indices are ordered chronologically (front is oldest)
2. **Monotonic values**: For max deque, `data[i1] > data[i2] > ... > data[ik]`
3. **Window bounds**: All indices satisfy `i > currentIndex - windowSize`
4. **Extrema property**: Front index always corresponds to current max/min

4.4 Query Operations

The `getMax()` and `getMin()` operations achieve $O(1)$ worst-case complexity through direct array access:

```java
public double getMax() {
    if (maxDeque.isEmpty()) {
        return Double.NEGATIVE_INFINITY;
    }
    return buffer[maxDeque.getFrontIndex()];
}

public double getMin() {
    if (minDeque.isEmpty()) {
        return Double.POSITIVE_INFINITY;
    }
    return buffer[minDeque.getFrontIndex()];
}
```

Listing 3: Constant-Time Query Operations

Performance Characteristics:

- **Time complexity**: $O(1)$ worst-case (no loops or recursion)
- **Cache efficiency**: Single array access with high locality
- **Branch prediction**: Predictable control flow aids CPU speculation

4.5 Dynamic Window Resizing

The `resize()` method supports runtime window size modification:

```java
public void resize(int newWindowSize) {
    if (newWindowSize <= 0) {
        throw new IllegalArgumentException("Window size must be
            positive");
    }

```

```java
    double[] newBuffer = new double[newWindowSize];

    // Determine range of valid elements to preserve
    int start = Math.max(0, currentIndex - windowSize);
    int elementsToKeep = Math.min(currentIndex - start, newWindowSize);

    int newIndex = 0;
    for (int i = currentIndex - elementsToKeep; i < currentIndex; i++) {
        newBuffer[newIndex++] = buffer[i % windowSize];
    }

    this.buffer = newBuffer;
    this.windowSize = newWindowSize;
    this.currentIndex = newIndex;

    maxDeque.clear();
    minDeque.clear();

    for (int i = 0; i < currentIndex; i++) {
        maxDeque.push(i);
        minDeque.push(i);
    }
}
```

Listing 4: Dynamic Window Resizing Algorithm

Complexity Analysis:

- **Buffer allocation**: $O(\text{newW})$
- **Copy operation**: $O(\min(W, \text{newW}))$
- **Deque reconstruction**: $O(\min(W, \text{newW}))$
- **Total**: $O(\text{newW})$

4.6 Memory Layout and Cache Considerations

The implementation is designed for optimal cache utilization:

Cache Optimization Strategies:

1. **Contiguous allocation**: Circular buffer uses single array allocation
2. **Spatial locality**: Sequential access patterns during buffer writes
3. **Temporal locality**: Recent indices accessed frequently
4. **Prefetching**: Predictable access enables hardware prefetching

Table 3: Memory Layout Analysis

Component	Size (bytes)	Access Pattern	Cache Friendly
Circular buffer	$8W$	Sequential	Yes
Max deque	$\leq 8W$	Front/back only	Yes
Min deque	$\leq 8W$	Front/back only	Yes
Total	$\leq 24W$	Mostly sequential	Yes

5 Video of the Implementation Running

5.1 Video Access Information

Zoom Link: *[To be provided]*

Password: *[To be provided]*

Comments: The video demonstrates the sliding window analytics engine processing various data streams, showing the performance comparison between the naive and optimized implementations, as well as dynamic window resizing capabilities.

6 References

References

[1] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. *Apache Flink: Stream and Batch Processing in a Single Engine.* IEEE Data Engineering Bulletin, 38(4):28–38, 2015.

[2] S. Muthukrishnan. *Data Streams: Algorithms and Applications.* Foundations and Trends in Theoretical Computer Science, 1(2):117–236, 2005.

[3] Robert E. Tarjan. *Amortized Computational Complexity.* SIAM Journal on Algebraic and Discrete Methods, 6(2):306–318, 1985.

[4] Trishul M. Chilimbi, Bob Davidson, and James R. Larus. *Cache-Conscious Structure Definition.* ACM SIGPLAN Notices, 34(5):13–24, 1999.

[5] Irene Aldridge. *High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems.* John Wiley & Sons, 2nd edition, 2013.

[6] Hao Zhang, Zehua Guo, Richard Watson, and Yuri Katchalski. *Real-Time Analytics and Stream Processing in IoT Applications.* In *IEEE International Conference on Big Data*, pages 2854–2863, 2018.

[7] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009.

[8] Donald E. Knuth. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Addison-Wesley, 3rd edition, 1997.

[9] Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser. *Data Structures and Algorithms in Java*. John Wiley & Sons, 6th edition, 2014.